

PROYECTO J & R

Métodos Numéricos con Python

MANUAL TÉCNICO

Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información

Curso: Métodos Numéricos — Python | Código: 021

Versión 1.0 — 2026

1. Arquitectura del Sistema

1.1 Descripción General

La aplicación sigue el patrón de diseño **MVC (Modelo-Vista-Controlador)** adaptado a PyQt5. La lógica de cada algoritmo está completamente separada de la interfaz gráfica, lo que facilita el mantenimiento y la extensión del sistema.

1.2 Estructura de Directorios

```
metodos_numericos/
■ ■ ■ main.py # Punto de entrada
■ ■ ■ requirements.txt # Dependencias
■ ■ ■ metodos/ # Módulo de algoritmos (Modelo)
■ ■ ■ ■ raices.py # Bisección, Newton-Raphson, Secante
■ ■ ■ ■ interpolacion.py # Lagrange, Newton
■ ■ ■ ■ sistemas.py # Gauss-Seidel, Factorización LU
■ ■ ■ ■ derivacion_integracion.py # Derivadas, Trapecio, Simpson
■ ■ ■ ■ ecuaciones_diferenciales.py # Euler, RK4
■ ■ ■ ui/ # Módulo de interfaz (Vista)
■ ■ ■ ■ main_window.py # Ventana principal con header y tabs
■ ■ ■ ■ tab_raices.py # Pestaña Módulo 1
■ ■ ■ ■ tab_interpolacion.py # Pestaña Módulo 2
■ ■ ■ ■ tab_sistemas.py # Pestaña Módulo 3
■ ■ ■ ■ tab_integracion.py # Pestaña Módulo 4
■ ■ ■ ■ tab_edo.py # Pestaña Módulo 5
■ ■ ■ themes/ # Módulo de temas visuales
■ ■ ■ theme_manager.py # Singleton que aplica el QSS
■ ■ ■ palettes.py # 5 paletas de colores definidas
```

1.3 Flujo de Datos

El flujo de ejecución típico es: el usuario ingresa parámetros en la **Vista** (tab_*.py), el Controlador integrado en la misma pestaña llama al **Modelo** (metodos/*.py), obtiene los resultados y los pasa a matplotlib para renderizar la gráfica en el panel derecho. El **ThemeManager** actúa como capa transversal que inyecta el stylesheet a toda la app en cualquier momento.

2. Especificaciones Técnicas

2.1 Tecnologías Utilizadas

3. Algoritmos Implementados

3.1 Módulo de Cálculo de Raíces

Bisección

Método de intervalo que divide repetidamente $[a,b]$ a la mitad. Garantiza convergencia si $f(a) \cdot f(b) < 0$. Convergencia lineal con tasa $1/2$.

$$c = (a + b) / 2 \quad | \quad \text{error} = |b - a| / 2$$

El intervalo se actualiza: si $f(a) \cdot f(c) < 0$ entonces $b=c$, si no $a=c$. El algoritmo termina cuando el error es menor que la tolerancia especificada.

Newton-Raphson

Método de convergencia cuadrática que utiliza la derivada de la función para calcular la siguiente aproximación. Requiere un punto inicial cercano a la raíz.

$$x_{n+1} = x_n - f(x_n) / f'(x_n)$$

El método falla si $f'(x)=0$. Requiere que el usuario suministre la expresión de la derivada $f'(x)$ manualmente.

Método de la Secante

Aproxima la derivada usando dos puntos anteriores, sin necesidad de la expresión analítica de $f'(x)$. Convergencia superlineal (orden ≈ 1.618).

$$x_{n+1} = x_n - f(x_n) \cdot (x_n - x_{n-1}) / (f(x_n) - f(x_{n-1}))$$

3.2 Módulo de Interpolación

Polinomio de Lagrange

Construye el polinomio interpolador de grado $n-1$ a partir de n puntos mediante la combinación lineal de los polinomios base $L_i(x)$:

$$P(x) = \text{SUM} [y_i * L_i(x)] \quad \text{donde} \quad L_i(x) = \text{PROD} [(x-x_j)/(x_i-x_j)] \quad (j \neq i)$$

Polinomio de Newton (Diferencias Divididas)

Forma alternativa del mismo polinomio interpolador, más eficiente para agregar puntos nuevos. Usa la tabla de diferencias divididas:

$$P(x) = f[x_0] + f[x_0, x_1](x-x_0) + f[x_0, x_1, x_2](x-x_0)(x-x_1) + \dots$$

3.3 Módulo de Sistemas de Ecuaciones

Gauss-Seidel

Método iterativo para sistemas lineales $Ax=b$. En cada iteración actualiza cada variable usando los valores más recientes disponibles:

$$x_i^{(k+1)} = (b_i - \text{SUM}[A_{ij} \cdot x_j]) / A_{ii} \quad (j \neq i)$$

Converge cuando la matriz A es diagonalmente dominante. El criterio de paro es la norma infinita del vector de diferencias entre iteraciones.

Factorización LU con Pivoteo Parcial

Descompone $A = P \cdot L \cdot U$ donde P es matriz de permutación, L es triangular inferior con 1s en la diagonal y U es triangular superior. Resuelve en dos pasos: sustitución hacia adelante ($Ly=Pb$) y hacia atrás ($Ux=y$).

3.4 Módulo de Derivación e Integración

Diferencias Finitas

Cuatro variantes implementadas:

- **Adelante:** $f'(x) \approx [f(x+h) - f(x)] / h$ — Error $O(h)$
- **Atrás:** $f'(x) \approx [f(x) - f(x-h)] / h$ — Error $O(h)$
- **Central:** $f'(x) \approx [f(x+h) - f(x-h)] / 2h$ — Error $O(h^2)$
- **Segunda:** $f''(x) \approx [f(x+h) - 2f(x) + f(x-h)] / h^2$ — Error $O(h^2)$

Regla del Trapecio Compuesta

$$\text{integral} \approx h/2 \cdot [f(x_0) + 2f(x_1) + \dots + 2f(x_{n-1}) + f(x_n)] \quad h=(b-a)/n$$

Regla de Simpson 1/3 Compuesta

$$\text{integral} \approx h/3 \cdot [f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \dots + f(x_n)]$$

Nota: n debe ser par. La implementación ajusta n automáticamente si se ingresa un valor impar.

3.5 Módulo de Ecuaciones Diferenciales

Método de Euler

Método de un paso de primer orden para resolver $y'=f(x,y)$ con condición inicial $y(x_0)=y_0$. Avanza con paso h:

$$y_{n+1} = y_n + h \cdot f(x_n, y_n)$$

Error de truncamiento local $O(h^2)$, error global $O(h)$. Útil para pasos pequeños; pierde precisión con h grande.

Runge-Kutta de Orden 4 (RK4)

Método de cuatro etapas con error global $O(h^4)$. Significativamente más preciso que Euler con el mismo paso h:

$$k_1=f(x_n, y_n) \quad k_2=f(x_n+h/2, y_n+h/2 \cdot k_1)$$

$$k_3 = f(x_n + h/2, y_n + h/2 \cdot k_2) \quad k_4 = f(x_n + h, y_n + h \cdot k_3)$$

$$y_{n+1} = y_n + (h/6) \cdot (k_1 + 2k_2 + 2k_3 + k_4)$$

4. Sistema de Temas Visuales (themes/)

4.1 Arquitectura del ThemeManager

El módulo `themes/` implementa el patrón **Singleton** para gestionar el tema visual de toda la aplicación desde un único punto de control:

- **palettes.py** — diccionario con las 5 paletas. Cada paleta define ~30 claves de color: fondos, textos, acentos, bordes y colores de matplotlib.
- **theme_manager.py** — clase `ThemeManager` (Singleton) que genera el QSS completo a partir de la paleta activa y lo aplica con `app.setStyleSheet()`.

4.2 Flujo de Cambio de Tema

Cuando el usuario selecciona una paleta en el diálogo de temas:

1. `ThemeDialog` llama a `ThemeManager().apply('Nombre')`
2. `ThemeManager` genera el QSS completo con `_build_qss(palette)`
3. `app.setStyleSheet(qss)` — aplica a TODOS los widgets de una sola vez
4. Se notifica a los callbacks registrados (e.g. `header gradient`)
5. Las gráficas usan `plot_colors()` en el siguiente render

4.3 Agregar una Nueva Paleta

Para extender el sistema con una nueva paleta, solo se edita `palettes.py`:

```
PALETTES['Mi Tema'] = {  
    'label': '■ Mi Tema',  
    'bg_main': '#0a1a0f',  
    'accent': '#22c55e',  
    # ... resto de claves  
}
```

Nota: No se requieren cambios en `main_window.py` ni en los tabs. El diálogo de temas se actualiza automáticamente.

5. Decisiones de Diseño

5.1 Separación Modelo-Vista

Todos los algoritmos residen en metodos/ como funciones puras sin dependencias de PyQt5. Esto permite probarlos de forma independiente y reutilizarlos en otros contextos (scripts de consola, Jupyter, etc.).

5.2 Paneles con QScrollArea

Los paneles de configuración usan QScrollArea en lugar de altura fija, lo que permite sistemas de $n \times n$ hasta 10×10 sin que los elementos se corten visualmente.

5.3 Altura Dinámica en Tablas

La Matriz A y el Vector b en el módulo de Sistemas calculan su altura exacta como $n \times \text{ROW_H} + \text{HEADER_EXTRA}$ pixeles, eliminando el scroll interno en las tablas y mostrando siempre todas las filas.

5.4 eval() Controlado para Funciones

Las funciones matemáticas $f(x)$ se evalúan con eval() pero dentro de un namespace controlado que solo expone numpy y funciones seguras (sin acceso a `__builtins__` completo). Toda evaluación está envuelta en try/except para mostrar errores claros al usuario.

5.5 Limitaciones Conocidas

- Gauss-Seidel puede no converger para matrices no diagonalmente dominantes.
- El rango de sistemas está limitado a 10×10 para mantener usabilidad de la UI.
- Las funciones ingresadas por el usuario deben usar sintaxis Python (`**` para potencia).
- El cambio de tema no actualiza colores en gráficas ya renderizadas; se actualizan en el siguiente cálculo.

6. Referencias Bibliográficas

- Chapra, S. C. (2007). *Métodos Numéricos para Ingenieros* (5ta ed.). McGraw-Hill.
- Burden, R. L. & Faires, J. D. (2002). *Análisis Numérico* (7ma ed.). Paraninfo.
- Lutz, M. (2013). *Learning Python* (5ta ed.). O'Reilly Media.
- Van der Walt, S. et al. (2011). The NumPy array. *Computing in Science & Engineering*.
- Documentación oficial PyQt5: <https://www.riverbankcomputing.com/software/pyqt/>
- Documentación oficial Matplotlib: <https://matplotlib.org/stable/>
- Documentación oficial NumPy: <https://numpy.org/doc/>